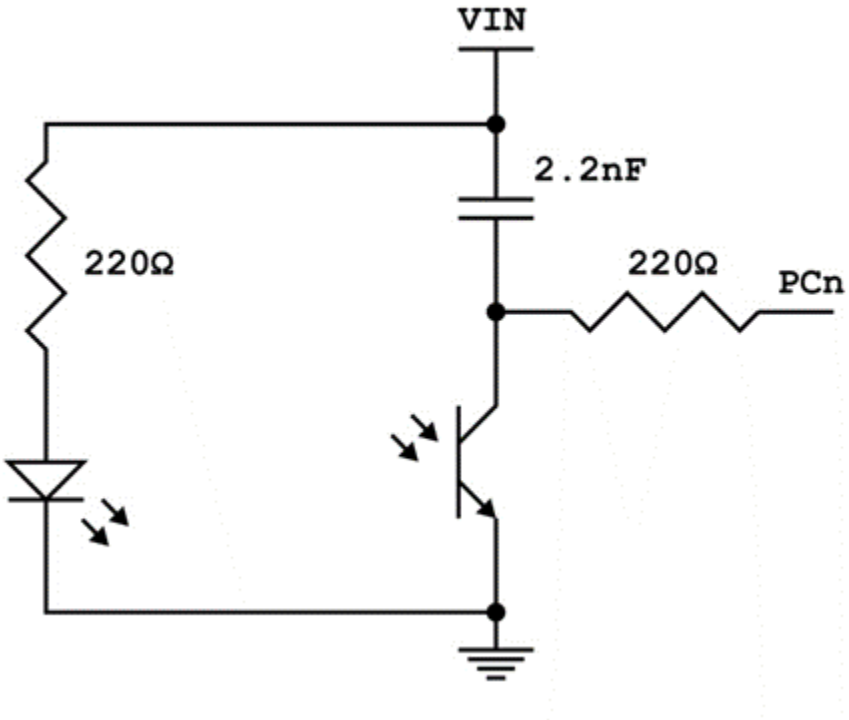
ECE 3218 Lab #8 - Controlling and Reading Volatile Sensors

Introduction

In this lab you will program the Pololu 3pi+ robot with AVR GCC using Microchip Studio. The goal is to set up and to read from the robot's array of 5 IR reflectance sensors. In Lab 10 and the final project you will combine this sensing with motor control from Lab 8 to enable the robot to follow lines. Consider carefully how you write these routines to make them as easy as possible to reuse.



QTR-1RC Reflectance Sensor

The figure above is a schematic of a single IR reflectance sensor on the 3PI robot. The sensor contains both an IR LED as a source and a photosensitive transistor as a sensing element. The source illuminates the surface immediately below it. The sensing element responds to the amount of light reflected by the surface. The sensor is called "volatile" because it must be initialized before every reading. The phototransistor is connected to a voltage source in series with a capacitor. To initiate a reading the capacitor is charged. Then charge flows from the capacitor through the phototransistor at a rate proportional to its illumination by light reflected from the surface. When the surface reflectance is high, much of the light from the LED illuminates the base of the transistor which causes the collector-emitter conductance to be high.

When the surface is dark, the illumination is low, and so is the conductance. As a result, the time it takes for the capacitor to discharge is a measure of the surface's brightness. Thus, the sensor is used by charging its capacitor then timing its discharge.

Each of the five sensors is connected as shown above through a resistor to one of the ATMega32U4's pins as shown in the table below:

Sensor	Port
DN1 (left)	PD6
DN2	PF7
DN3 (center)	PF5
DN5	PF4
DN5 (right)	PF1

Also you will need to control the IR emitter. It is connected to PB7.

The sensor circuit uses the ability of the AVR microcontroller to reconfigure its digital inputs as digital outputs on-the-fly. A digital output presents a voltage of 5 V or 0 V, depending on whether it is set to a 1 or a 0 by the program. At the start of a sensing cycle, the pin is set to a 5V output for 10μS which allows the capacitor to charge. Then the pin is switched to input. That input reads 1 until the capacitor has discharged through the phototransistor, at which time the input changes to 0. This can be detected by polling the pin values or setting up pin change interrupts on each of the sensor pins. If a timer is set up to start when the capacitor is charged, and to stop upon a pin change, the timer reading will indicate the relative brightness of the underlying surface.

Example code that controls and reads from sensor 0 is given below. Your objective is to set up all the sensors so that they can be read in sequence. A 5-element array is to be kept that indicates the brightness of the surface under each sensor. This array will be a central component in the control of line-following by the robot.

Before the Lab

Part 1: Understand how the Pololu's IR sensors work and the steps necessary to control them.
Read through the corresponding sections in the *Pololu 3pi+ User's Guide*. Describe the steps in detail. Record them in your notebook.

Part 2: Write a program to control and to read from all five sensors.

There are any number of ways to do this. For example, each sensor could be started then read in succession, or all the sensors could be started together and read individually as the associated pin changes. Below is an algorithm for starting the sensors together and reading them as they change polarity. Also, there is example code that implements the one-at-a-time approach but includes only one of the five sensors. You may use it but you will have to modify it to read all five sensors. The method it implements is inefficient. The algorithm describes a much better way to read the sensors. We suggest you write a program that implements the algorithm and refer to the code for examples.

We used Timer1 for the motor control. Therefore, we recommend using Timer3 for measuring the sensors.

Make sure to give some thought to the logic about how the pin changes are recorded. There are easy ways, difficult ways, and clumsy ways.

Part 3: Design experiments to:

1. Determine the average timeout periods of each of the 5 sensors over bright and dark surfaces and
2. Estimate the angle of the robot with respect to a 1cm wide black line on a white background under the robot.

During the Lab

Implement and test the program from part 2. Perform the experiments in part 3 and record the results in your notebook.

After the Lab

Make listings of your corrected, annotated programs. Comment on any unexpected behaviors exhibited by the sensors during the experiments. Describe and comment on the results of Part 3 above.

Algorithm for reading the Pololu 3Pi sensors

Setup and main loop:

1. Initialize your port directions and values
2. Initialize and start a timer
3. Turn off the IR emitter LED
4. Set the sensor pins to be outputs and set them high to charge the capacitors
5. Wait 10us and change them to inputs (why?)
6. Disable pullups (will keep capacitors from discharging)
7. Wait for each sensor to change – polling or interrupts

8. Record the time it takes for each to change
9. After all the pins have changed, or a preset maximum time has elapsed, restart the process

Example Program to read one sensor

```
/*
* ReadSensorSampleCode.c
*
* Sample code to read from a single sensor and write the output to the OLED
* Author : sternbal
*/


#include <avr/io.h>
#include "OLED.h"


#define EMIT PB7 // IR Emitter
#define DN1 PD6 // Down sensor 1


#define F_CPU 16000000UL
#include <util/delay.h>


int main(void)
{
    uint16_t sensorTime;
    char text[20];
    // Initialize the OLED display
    OLED_init();
    OLED_reset();
    OLED_clearDisplayRam();
    OLED_configureDefault();


    // setup the emitter pin
    DDRB |= 1<<EMIT;


    // setup and start Timer3
    TCCR3A = 0x00;
    TCCR3B = 0x02;

        while (1)
        {
                    // charge the capacitor by setting the pin to an
                // output and making the value 1
                    DDRD |= 1<<DN1;
                    PORTD |= 1<<DN1;


                    // turn the emitter off while we're charging
                    PORTB &= ~(1<<EMIT);
                    // wait 10 us
                    _delay_us(10);

                    // reset the timer
                    TCNT3 = 0;
```

```
                        // set pin to input
                        DDRD &= ~(1<<DN1);


                        // make sure the pull-up resistor is disabled
                        PORTD &= ~(1<<DN1);


                        // turn the emitter on
                        PORTB |= 1<<EMIT;

                        // loop while the pin is still 1.
                        while(TCNT3<20000) // gives it a maximum timeout value.
                        {
                               if((PIND & (1<<DN1)))
                               {
                                      sensorTime=TCNT3;
                               }
                        }
                        // sensorTime now holds the value of the timer when it changed.
                        snprintf(text,10,"%i",sensorTime);
                        OLED_clearDisplayRam();
                        OLED_writeText(text);
                }
        }
```